Aegis-Split

Escrow marketplace: clients fund milestones via PayPal, freelancers deliver on GitHub, and an AI auditor verifies work before payout.

1. The problem

Freelance and contract work often fails on trust. Clients worry about paying upfront and getting ghosted or low-quality deliverables. Freelancers worry about finishing work and never being paid — chasing invoices, disputing chargebacks, or learning that the client disappeared after handoff.

Traditional invoicing does not tie payment to proof of completion. Escrow helps, but manual escrow is slow and opaque. Aegis-Split automates the loop: agreed milestones, locked funds, verifiable delivery on GitHub, machine-assisted review against acceptance criteria, and release only when checks pass.

Who it helps: Product teams hiring contractors, agencies with milestone billing, and individual freelancers who want payment tied to objective delivery signals rather than email promises.

2. How escrow works

1. Agree — Client and freelancer create a project with milestones — task, acceptance criteria, and amount.

2. Verify — Both parties complete identity verification and monitoring consent (required before funding or linking code).

3. Fund — Client pays via PayPal Orders API. Funds are captured into the vault; money is locked, not paid out yet.

4. Build — Freelancer links a GitHub repository and pull request to the milestone, then merges when ready.

5. Audit — On merge, Aegis-Split fetches the PR diff, runs the AI auditor against acceptance criteria, and records pass or fail.

6. Pay — If the audit passes, PayPal Payouts releases that milestone’s amount to the freelancer; both sides get notifications.

The same cycle repeats for each milestone until the project is complete. Funds never leave the vault for a milestone until its audit passes (or you intervene manually in demo/admin flows).

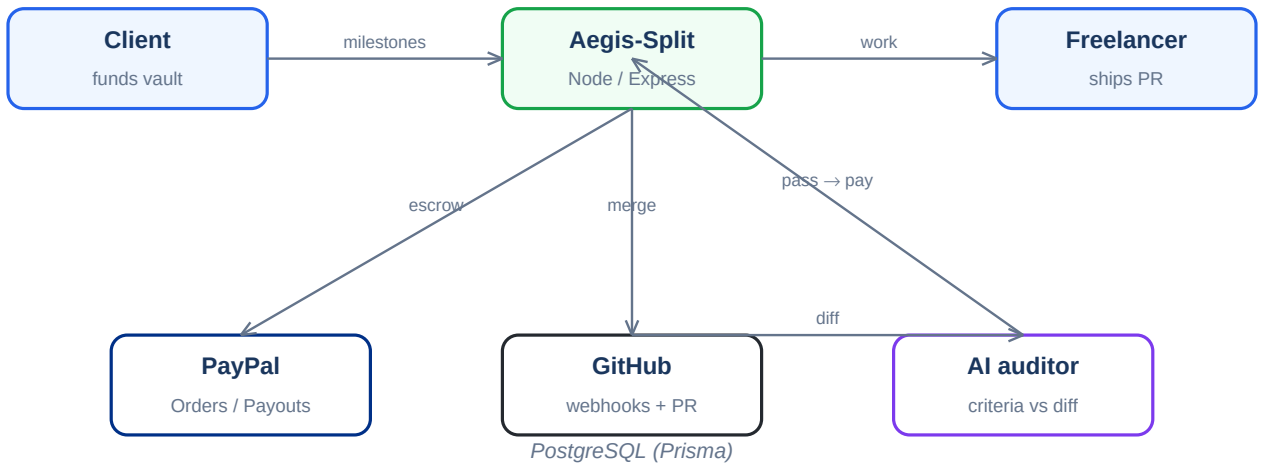
3. Tech stack

Runtime & API	Node.js 18+, Express — REST API and static demo UI on the same origin
Data	Prisma ORM + PostgreSQL (e.g. Neon on Render)
Payments	PayPal REST — Orders API (fund / capture) and Payouts API (release)
Source control	GitHub — repo link per milestone, HMAC-verified pull_request webhooks
Verification	AI auditor — LLM when API keys are set; deterministic heuristics otherwise
Identity	DigiLocker / Stripe Identity scaffolds; consent logging with KYC gates
Hosting	Docker image; Render Blueprint (preferred), Railway, or Fly.io — live at aegis-split.onrender.com

4. Architecture

Browser clients talk to the Aegis-Split app. The app orchestrates PayPal escrow, listens for GitHub merge events, invokes the auditor, and persists state in PostgreSQL.

System architecture (high level)



Key API areas: `/api/vault/*` (fund/capture), `/api/webhooks/github` (merge), `/api/projects/*/milestones/*` (milestones, link, release), `/api/notifications` (FUNDED, AUDIT_*, PAID).

Trust & safety (before money moves)

Before creating projects, funding a vault, or linking production merges, each user must complete government-ID verification (DigiLocker for India, Stripe Identity scaffold elsewhere) and grant monitoring consent with a timestamped audit log. Middleware on protected routes returns `403` until both flags are set — reducing anonymous dispute risk without replacing PayPal as the payment rail.

Notifications: Idempotent in-app alerts fire for FUNDED, AUDIT_PASSED / AUDIT_FAILED, and PAID events so both client and freelancer see the same timeline.

5. Live demo

Production sandbox deployment: **https://aegis-split.onrender.com**

The hosted app serves the demo UI at `/` and the Express API under `/api/*`. Use PayPal sandbox accounts and the demo seed flow to walk through fund → merge → audit → paid without installing locally.

6. How to test

Live (Render)

- Open https://aegis-split.onrender.com
- Click **Seed Sarah & Rohan** (uses demo admin key configured on the host)
- Switch user → complete gov ID verify + monitoring consent (sandbox/demo mode as deployed)
- Create a project and milestone → **Fund vault** → approve in PayPal sandbox (or mock if enabled)
- Link GitHub repo → **Simulate merge** (or trigger a real webhook if the app URL is public)
- Confirm AI audit PASS, milestone PAID, and in-app notifications

Local

```
cp .env.example .env && npm install && npx prisma migrate deploy && npm start
```

Open `http://localhost:3000`. Auth is stubbed: the demo UI sets `X-User-Id` when you pick a user.

Environment variables (high level — see `.env.example`):

`DATABASE_URL`	PostgreSQL connection string
`PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET`	Sandbox credentials (or PAYPAL_MOCK=true)
`PAYPAL_MODE`	sandbox
`GITHUB_WEBHOOK_SECRET / GITHUB_TOKEN`	For real webhooks and PR diffs; optional for simulate-merge
`DEMO_ADMIN_KEY`	Defaults to hackathon-demo-key for seed and simulate-merge
`OPENAI_API_KEY or ANTHROPIC_API_KEY`	Optional — enables LLM auditor; otherwise deterministic

Run `npm test` for KYC, PayPal, GitHub, auditor, and notification unit checks. For full payout testing without PayPal credentials, set `PAYPAL_MOCK=true`.

API walkthrough (local or curl)

After seeding users, pick a user in the UI (sets `X-User-Id`) or pass the header manually:

```
POST /api/vault/fund { projectId, milestoneId, amountCents }

GET /api/vault/paypal/return?token=ORDER_ID (after PayPal approve)

POST /api/projects/:projectId/milestones/:id/github { githubRepoFullName,
githubPrNumber }

POST /api/admin/demo/milestones/:id/simulate-merge (+ X-Demo-Admin-Key)

POST /api/projects/:projectId/milestones/:id/release { recipientEmail }
```

Real GitHub webhook: Expose your app URL (Render fulfills this), set `GITHUB_WEBHOOK_SECRET` on the repo webhook, and merge a linked PR. The handler verifies HMAC, pulls the diff with `GITHUB_TOKEN`, runs the auditor, and on PASS calls the same release path as simulate-merge.

PayPal sandbox notes: Default flow creates an order and returns an `approvalUrl` for the sandbox buyer. Capture runs on `/api/vault/paypal/return` when the order is APPROVED. Set the freelancer user's `paypal_email` to a sandbox personal account for payout testing.